

Liquify your Programs

Rajarshi Goswami¹ and Ashish Mishra¹

IIT Hyderabad, India

cs24mtech12012@iith.ac.in, mishraashish@cse.iith.ac.in

1 Introduction

Traditional type systems prevent basic errors but lack the expressiveness to handle logical errors or prove complex program properties. Refinement types [5] address this by augmenting base types with decidable first-order logic constraints. However, writing refinement types manually is difficult [1], hindering their wider adoption. Existing symbolic [5] and data-driven [8, 9] inference approaches often struggle to scale to multi-function programs or rely heavily on manual annotations.

We present Elixir, a neurosymbolic framework for automated refinement type synthesis. Figure 1 gives an overview for Elixir. Elixir encodes refinement typing rules into a novel representation, called Liquid Graph and learns semantic representations via a Graph Neural Network (GNN). An autoregressive model then synthesizes predicates, using Reinforcement Learning (RL) to navigate the search space while a type checker verifies candidates.

A core challenge in this setting is the sparse-reward problem [3], where type checker feedback provides insufficient guidance for the RL agent. To address this, we introduce two main contributions: (i) Novel Behavioral Rewards utilizing subspecifications [4] and semantic program slicing [2, 7] to provide partial rewards. (ii) A data-driven, CEGAR-Guided Learning loop that refines reward signals based on incremental correctness.

Together, these techniques generate denser rewards and accelerate convergence. Our implementation and initial evaluation of Elixir efficiently generates semantically correct specifications for complex programs and properties. A detailed evaluation is ongoing.

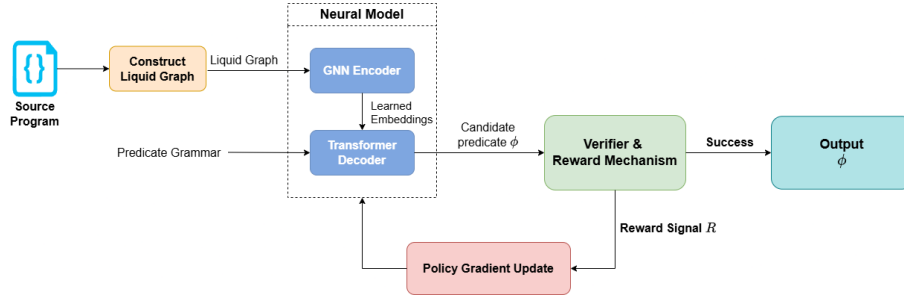


Fig. 1. Neurosymbolic RL refinement type synthesis architecture.

2 Elixir Details

2.1 Liquid Graphs

We encode program refinement semantics into a graph that mirrors the standard typing-derivation process, albeit, with variables corresponding to the missing refinement types and predicates. Nodes in Liquid Graph represent program expressions, variables, and placeholders for unknown refinement predicates. Meta-nodes capture environment extensions (for branching), function applications, and expression ordering. Semantic relationships are encoded via different categories of edges: expression edges connect expressions, variable edges connect expressions to variables, type edges associate program variables with their refinement type variables, and refinement edges connect variables to unknown predicates.

2.2 Specification Inference and Dense Reward Calculation

Specification generation is handled by a transformer-based, syntax-constrained decoder over a predicate grammar. When candidates fail verification, an RL loop updates the model parameters to iteratively refine the predicates. To address the sparse reward problem in this setting, we use three main ideas for guiding RL reward generation:

Subspecification : We introduce partial rewards for sub-specifications [4]. For example, consider the specification $\nu = x \wedge \nu > y$ for the function $\mathbf{max} \ x \ y = \mathbf{if} \ x > y \ \mathbf{then} \ x \ \mathbf{else} \ y$, which the verifier rejects. We instead decompose the specification, verify each independently against the program branches, and other sub-terms, and assign a reward proportional to those satisfied. For instance, the sub-specification $\nu \geq x$ holds for the **then**-branch, which returns x . Formally, let $\mathcal{E}_e(e)$ denote the set of sub-expressions of e , and $\mathcal{E}_\phi(\phi)$ the set of sub-specifications for ϕ . We define an indicator function, $\mathbf{1}[\text{Check}(e, \phi)]$, evaluating to **1** if sub-expression e type-checks under sub-specification ϕ . The reward is defined as:

$$R_{sub}(\phi) = (1 - V(\phi)) \cdot \frac{1}{|\mathcal{E}_\phi(\phi)|} \sum_{\phi_i \in \mathcal{E}_\phi(\phi)} \max_{e_i \in \mathcal{E}_e(e)} \mathbf{1}[\text{Check}(e_i, \phi_i)]$$

where $V(\phi) \in \{0, 1\}$ denotes the verifier outcome, with success (1) or failure (0).

Semantic Slicing: We use semantic slices of the program with respect to each formal argument to guide partial reward generation. By sampling inputs along a static slice, we generate concrete executions and record the corresponding outputs. For example, the specification $\nu = x$ for the function $\mathbf{max}$ captures the behavior along only the **x**-slice. Including executions along the **y**-slice acts as a partial reward to guide the agent towards a complete specification. Let $B = \{b_1, \dots, b_k\}$ denote these observed behaviors, where each b_i is an input-output tuple. Using an indicator $\mathbf{1}[\phi \models b_i]$ that evaluates to 1 if behavior b_i is consistent with ϕ , we define the behavioral reward as:

$$R_{slice}(\phi) = \frac{1}{|B|} \sum_{b_i \in B} \mathbf{1}[\phi \models b_i]$$

Counter-Examples: We also use counter-examples in the style of a CEGAR loop. By accumulating counterexamples from previously failed attempts, candidates receive a reward proportional to the fraction of counterexamples they satisfy, and the verifier is only invoked if all are passed. Given an accumulated set of counterexamples $CE = \{ce_1, \dots, ce_n\}$, we define an indicator $\mathbf{1}[\phi \models ce_i]$ which equals 1 if candidate ϕ satisfies ce_i . The reward is expressed as:

$$R_{ce}(\phi) = \frac{1}{|CE|} \sum_{ce_i \in CE} \mathbf{1}[\phi \models ce_i]$$

The final reward used to train the RL agent is:

$$R(\phi) = \alpha R_{sub}(\phi) + \beta R_{slice}(\phi) + \delta R_{ce}(\phi) + V(\phi)$$

where α, β , and δ are user-defined hyperparameters.

3 Implementation and Evaluation

We have implemented the above ideas in a prototype tool named Elixir. We have initially evaluated Elixir on a suite of arithmetic and benchmark OCaml programs from [9]. The results are summarized in table. 1. An extended evaluation is in progress on larger benchmarks.

4 Related Works and Conclusion

Liquid Types [5] provide limited refinement type inference by solving logical constraints over program variables. Dorder [8, 9] abstracts program behavior using data-driven techniques, while recent efforts prompt fine-tuned code models [6] for inference. In contrast, we propose the first neurosymbolic RL framework that bridges formal refinement semantics, subspecifications, and slicing with generative learning. Initial prototype testing demonstrates the feasibility of our dense reward mechanism. Future work will compare our approach against LLM-based specification inference and repair frameworks.

Table 1. Performance of Elixir on arithmetic and functional data structure programs.

Category	Benchmark	Success (%)	Best Inferred Specification
Arithmetic	max	100	$\nu \geq x$
	abs	100	$\nu \geq x$
	clamp	60	$a \geq \nu \parallel a < x$
	signum	100	$\nu \geq x \vee x \neq \nu$
Lists	list_append	100	$\forall u. \text{mem_list}(ys, u) \vee \text{mem_list}(xs, u) \implies \text{mem_list}(\nu, u)$
	alternate	100	$\forall u. \text{mem_list}(ys, u) \implies \text{mem_list}(\nu, u)$
Linked Lists	ll_append	80	$\text{len_ll_list}(\nu) - \text{len_ll_list}(xs) = \text{len_ll_list}(ys)$
BST	bst_insert	80	$\text{mem_bst}(\nu, x)$
	flatten	100	$\forall u. \text{mem_bst}(t, u) \implies \text{mem_list}(\nu, u)$

References

1. Gamboa, C., Reese, A., Fonseca, A., Aldrich, J.: Usability barriers for liquid types. *Proceedings of the ACM on Programming Languages* **9**(PLDI), 1911–1936 (2025)
2. Horwitz, S., Reps, T., Binkley, D.: Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems (TOPLAS)* **12**(1), 26–60 (1990)
3. Ladosz, P., Weng, L., Kim, M., Oh, H.: Exploration in deep reinforcement learning: A survey. *Information Fusion* **85**, 1–22 (2022)
4. Nazari, A., Huang, Y., Samanta, R., Radhakrishna, A., Raghothaman, M.: Explainable program synthesis by localizing specifications. *Proc. ACM Program. Lang.* **7**(OOPSLA2) (Oct 2023). <https://doi.org/10.1145/3622874>, <https://doi.org/10.1145/3622874>
5. Rondon, P.M., Kawaguchi, M., Jhala, R.: Liquid types. In: *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. p. 159–169. PLDI '08, Association for Computing Machinery, New York, NY, USA (2008). <https://doi.org/10.1145/1375581.1375602>, <https://doi.org/10.1145/1375581.1375602>
6. Sakkas, G., Sahu, P., Ong, K., Jhala, R.: Neurosymbolic modular refinement type inference. In: *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*. p. 846–858. ICSE '25, IEEE Press (2025). <https://doi.org/10.1109/ICSE55347.2025.00090>, <https://doi.org/10.1109/ICSE55347.2025.00090>
7. Weiser, M.: Program slicing. In: *Proceedings of the 5th International Conference on Software Engineering*. p. 439–449. ICSE '81, IEEE Press (1981)
8. Zhu, H., Nori, A.V., Jagannathan, S.: Learning refinement types. *ACM SIGPLAN Notices* **50**(9), 400–411 (2015)
9. Zhu, H., Petri, G., Jagannathan, S.: Automatically learning shape specifications. In: *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. pp. 491–507 (2016)